

Instituto Tecnológico de Costa Rica

Sede Central Cartago, Escuela de Ingeniería en Computación



Documentación del Diseño del Proyecto

Restaurantes E2

Curso:

Bases de Datos 2

Profesor:

Kenneth Obando Rodríguez

Estudiantes:

Joshua Corrales Retana (2024073529)

Felipe Lepiz Retana (2024800990)

Fecha de entrega:

07/05/2026

Tabla de contenido

1. Resumen ejecutivo.....	3
2. Objetivos del Proyecto.....	3
2.1 Objetivo general.....	3
2.2 Objetivos especificos	3
3. Alcance técnico.....	4
4. Arquitectura lógica.....	5
5. Arquitectura Física	6
6. Descripción de cada microservicio y sus responsabilidades.....	7
6.1 API Principal.....	7
6.1.1 Responsabilidades.....	8
6.1.2 Rutas principales	8
6.1.3 Middlewares.....	8
6.1.4 Configuración (apps/api/src/config/).....	9
6.1.5 Forma estándar de respuesta.....	9
6.1.6 Documentación interna OpenAPI	9
6.2 Microservicio de búsqueda y Elasticsearch.....	10
6.2.1 Responsabilidades.....	11
6.2.2 Endpoints	11
6.2.3 Modelo de índice	11
7. Descripción general del flujo de datos y operación.....	12
7.1 Uso del balanceador.....	12
7.1.1 Nginx Ingress y balanceo.....	13
7.1.2 Estrategia de routing	13
7.1.3 Diferencia entre Ingress y Ingress Controller	13
8. Representación del Cluster de Kubemetes.....	14

1. Resumen ejecutivo

Restaurantes E2 corresponde a la segunda etapa del Proyecto 1 del curso Bases de Datos 2. El sistema toma como base la API REST de gestión de restaurantes desarrollada en la primera etapa y la extiende hacia una arquitectura distribuida. Esta versión incorpora dos motores de base de datos intercambiables mediante configuración, PostgreSQL y MongoDB, un microservicio de búsqueda apoyado en Elasticsearch, una capa de caché con Redis, autenticación centralizada con Keycloak, enrutamiento y balanceo mediante NGINX Ingress, despliegue en Kubernetes y una canalización de integración y entrega continua con GitHub Actions.

El núcleo transaccional se mantiene como una API modular desarrollada con Express y TypeScript. La separación entre lógica de negocio y persistencia se implementa mediante interfaces DAO, lo que permite cambiar el motor activo con la variable `DB_ENGINE` sin modificar controladores ni servicios. La búsqueda se separa en un microservicio independiente para aislar la responsabilidad de consulta textual y mantener Elasticsearch como una proyección de lectura, no como fuente principal de datos.

El despliegue en Kubernetes se automatiza mediante scripts de PowerShell. Según el motor seleccionado, el sistema puede ejecutarse con un clúster MongoDB particionado de 10 pods o con una instancia única de PostgreSQL para la base de datos de negocio. Además, el proyecto cuenta con pruebas automatizadas y una validación de cobertura mínima del 90% antes de aceptar cambios.

2. Objetivos del Proyecto

2.1 Objetivo general

Diseñar y desplegar una arquitectura distribuida para una API de gestión de restaurantes, aplicando conceptos de Bases de Datos 2 como replicación, particionamiento, motores relacionales y documentales, motores de búsqueda, caché, autenticación centralizada, balanceo de carga y operación mediante contenedores.

2.2 Objetivos específicos

- Aislar la persistencia con un patrón DAO que permita seleccionar PostgreSQL o MongoDB sin modificar la lógica de negocio.
- Configurar un cluster MongoDB con replica sets y al menos un shard, particionando productos y reservas.
- Crear un microservicio de búsqueda con Elasticsearch que indexe los items del menu y exponga endpoints de consulta.
- Cachear las respuestas de lectura más frecuentes con Redis, aplicando una estrategia cache-aside con TTL y política de evicción.
- Integrar Keycloak como proveedor de identidad para emitir y validar JWT.
- Enrutamiento y balanceo con Nginx (Ingress en Kubernetes) entre la API y el microservicio de búsqueda.
- Definir un pipeline CI/CD que ejecute pruebas, construya imágenes Docker y publique en GHCR.
- Garantizar al menos 90% de cobertura de pruebas en los módulos clave.
- Documentar la arquitectura de forma defendible, con diagramas y decisiones técnicas explícitas.

3. Alcance técnico

Esta documentación describe el estado actual del sistema y los componentes implementados en el repositorio. Cada elemento arquitectónico se relaciona con archivos concretos del proyecto, con el fin de facilitar la revisión técnica, la defensa del diseño y la trazabilidad entre la documentación y la implementación.

Componente	Estado
API principal con todas las rutas (auth, users, restaurants, menus, reservations, orders)	Implementado
Patrón DAO con interfaces y dos implementaciones (Postgres / Mongo)	Implementado
PostgreSQL vía stored procedures (sp_*)	Implementado
MongoDB sharded cluster (configsvr + 2 shards + mongos)	Implementado
Mongo: sharding de menuitems y reservations	Implementado
Microservicio Search con Elasticsearch	Implementado
Re-indexación bajo demanda (POST /search/reindex)	Implementado
Redis cache-aside en GET frecuentes	Implementado
Autenticación vía Keycloak (JWKS, RS256)	Implementado
Roles client / restaurant_admin	Implementado
Docker Compose para entorno local completo	Implementado
Manifiestos Kubernetes para todos los componentes	Implementado
Ingress NGINX con routing /api y /search	Implementado
CI con lint, tests y gate de cobertura >= 90%	Implementado
CD con build y push a GHCR (api y search)	Implementado
ADRs (10 decisiones técnicas documentadas)	Implementado
Colección Postman de referencia	Implementado

4. Arquitectura Lógica

El sistema se organiza en una arquitectura lógica basada en servicios. El acceso externo se realiza mediante un único punto de entrada, NGINX Ingress, que enruta las solicitudes HTTP según el prefijo de la ruta: `/api` hacia la API principal y `/search` hacia el microservicio de búsqueda. La API principal concentra la lógica transaccional del sistema, incluyendo autenticación, usuarios, restaurantes, menús, reservas y pedidos. Esta API utiliza el patrón DAO para desacoplar la lógica de negocio del motor de base de datos, permitiendo operar con PostgreSQL o MongoDB según el valor configurado en `DB\_ENGINE`.

PostgreSQL 16 se utiliza como motor relacional con stored procedures, mientras que MongoDB 7 se despliega como un cluster sharded para el modo documental. Estos motores son alternativas de persistencia para la API principal; en Kubernetes se despliega solo uno como motor de negocio activo según el comando de despliegue. Redis 7 funciona como una capa de caché para reducir consultas repetitivas a la base de datos, y Keycloak 26 actúa como proveedor de identidad, emisión de tokens JWT y validación mediante JWKS.

El microservicio Search se mantiene separado de la API principal para aislar la responsabilidad de búsqueda full-text. Este servicio consulta Elasticsearch 8.13 como índice especializado de productos y, durante el proceso de reindexación, obtiene los ítems de menú desde la API principal mediante el endpoint interno `GET /api/menus/items/all`. De esta forma, Elasticsearch se mantiene como una proyección de lectura y no como fuente de verdad del sistema.

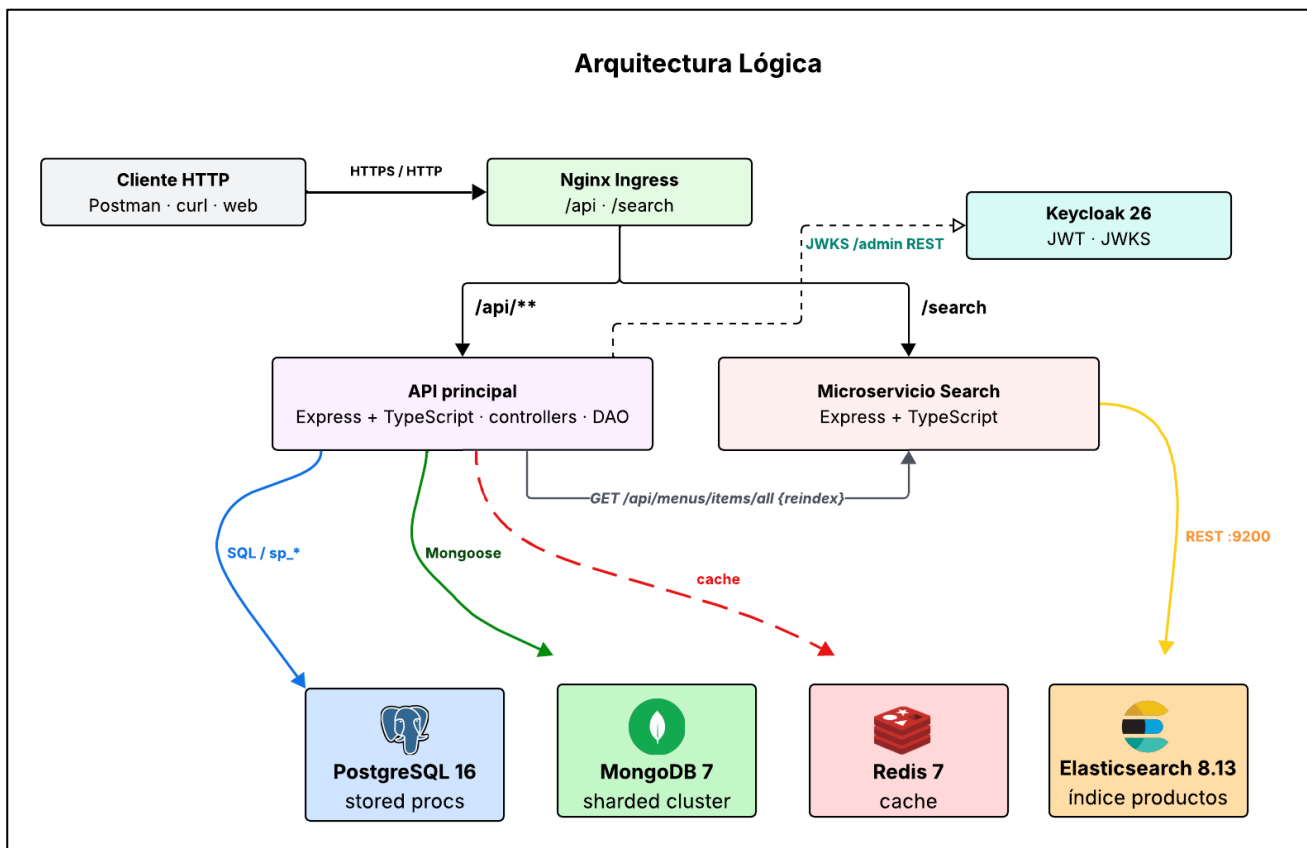


Figura 1. Arquitectura lógica del sistema y comunicación entre servicios principales.

5. Arquitectura Física

La arquitectura física del sistema se ejecuta sobre Kubernetes dentro del namespace proyecto01-restaurante. Los servicios sin estado, como la API principal y el microservicio de búsqueda, se despliegan como Deployments y se exponen internamente mediante servicios ClusterIP. El acceso externo se centraliza en un recurso Ingress, interpretado por NGINX Ingress Controller, que dirige el tráfico hacia /api o /search según el prefijo de la ruta.

Los componentes con almacenamiento persistente se despliegan mediante StatefulSets. MongoDB utiliza tres conjuntos de réplicas: uno para el servidor de configuración y dos para los shards de datos. PostgreSQL, cuando se selecciona como motor de negocio, se ejecuta como una instancia única con volumen persistente. Elasticsearch también usa un StatefulSet con almacenamiento propio, mientras que Redis se ejecuta como Deployment porque su información es efímera y puede reconstruirse desde la base de datos.

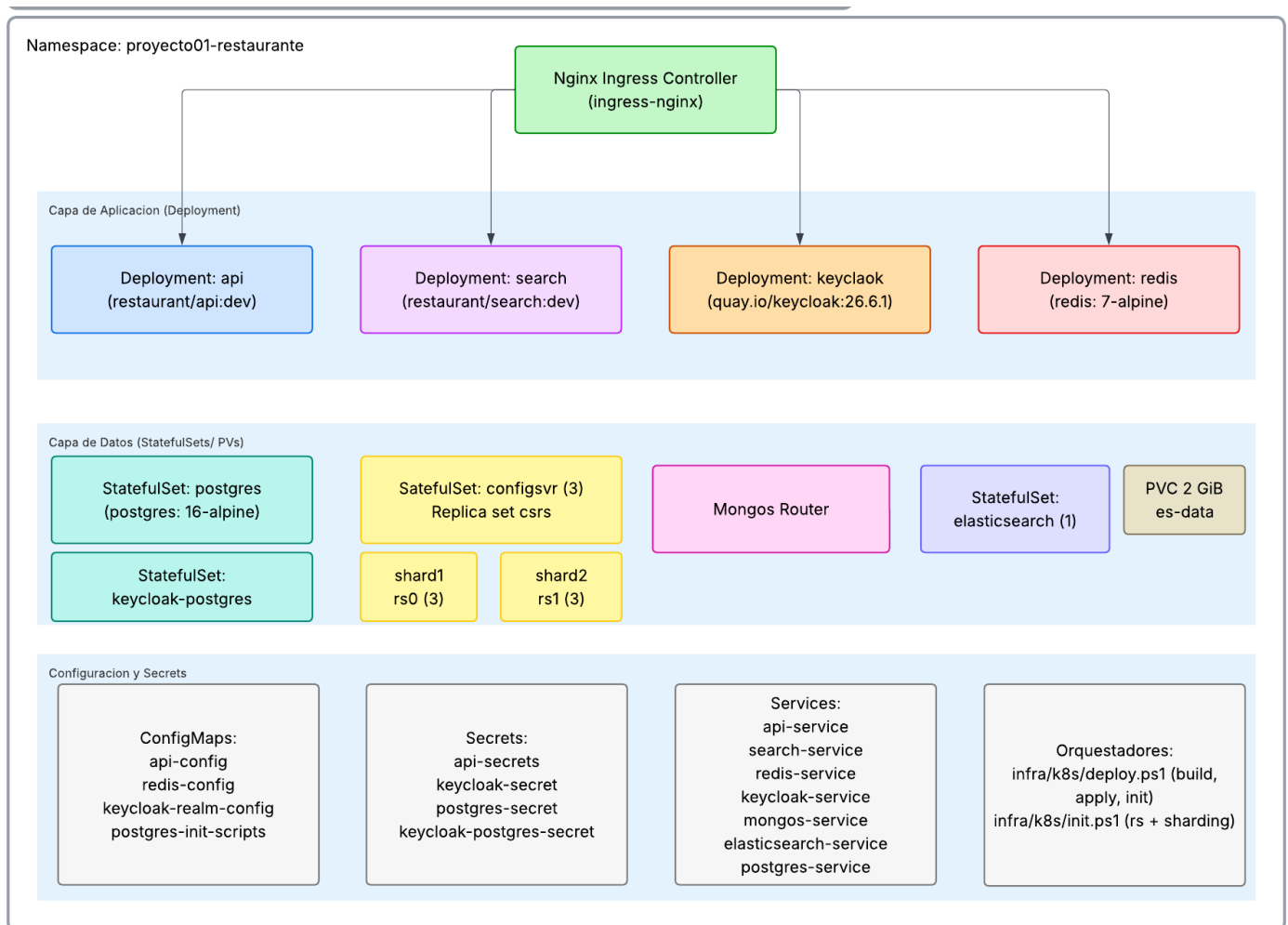


Figura 2. Arquitectura física del sistema desplegado en Kubernetes.

6. Descripción de cada microservicio y sus responsabilidades

6.1 API Principal

La API principal es el componente transaccional central del sistema. Está desarrollada con Express y TypeScript, y expone los recursos relacionados con autenticación, usuarios, restaurantes, menús, reservas y pedidos. Su responsabilidad consiste en recibir solicitudes HTTP, validar datos de entrada, aplicar autenticación y autorización, ejecutar la lógica de negocio y delegar la persistencia mediante el patrón DAO.

La API no accede directamente a la base de datos desde los controladores. Los servicios utilizan el objeto dao, construido por DaoFactory de acuerdo con la variable DB_ENGINE. Gracias a esta separación, la misma lógica de aplicación puede trabajar con PostgreSQL o MongoDB sin cambios en las capas superiores.

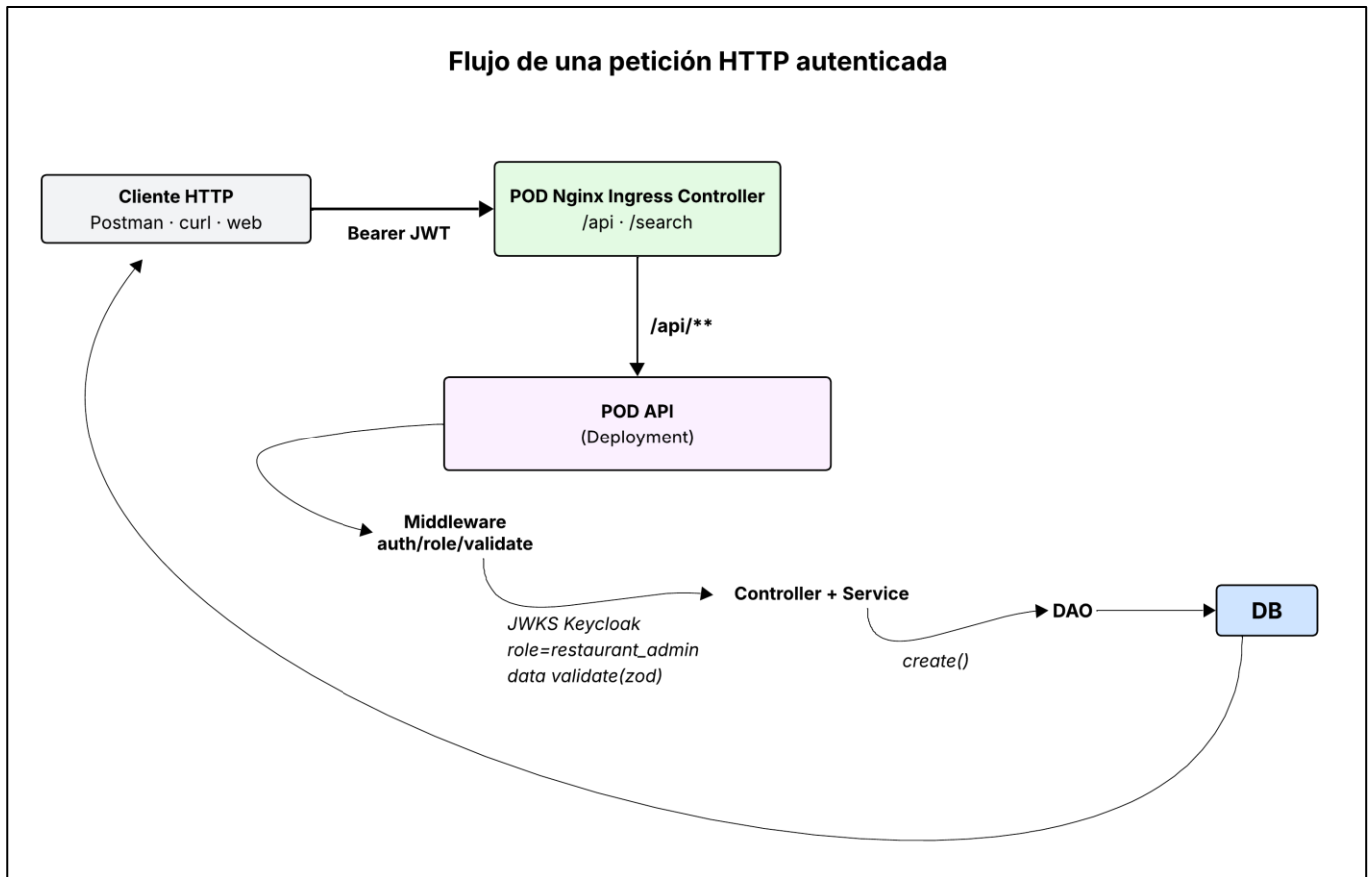


Figura 3. Flujo de una petición autenticada desde el cliente hasta la base de datos.

6.1.1 Responsabilidades

La API principal se encarga de exponer los endpoints transaccionales, aplicar validación de entrada, autenticar usuarios, autorizar operaciones según roles, ejecutar reglas de negocio y delegar el acceso a datos mediante interfaces DAO. También integra Keycloak para identidad y emisión de tokens JWT, utiliza Redis como caché para consultas frecuentes, selecciona el motor de datos activo mediante DB_ENGINE y mantiene una estructura uniforme de respuesta. Además, expone /health para monitoreo y /api-docs como documentación interactiva basada en OpenAPI.

6.1.2 Rutas principales

Recurso	Endpoints representativos	Middlewares aplicados
Auth	POST /api/auth/register, POST /api/auth/login	validate (Zod)
Users	GET /api/users/me, PUT /api/users/:id, DELETE /api/users/:id	authenticate
Restaurants	GET /api/restaurants, POST /api/restaurants	cacheMiddleware (120 s) en GET; authenticate + authorize('restaurant_admin') en POST
Menus	CRUD de menus e items (POST/PUT/DELETE). GET /api/menus/restaurant/:id, /menus/:id, /menus/:menuId/items	cacheMiddleware (60-120 s); authenticate + authorize('restaurant_admin') en escrituras
Reservations	POST /api/reservations, GET /me, /restaurant/:id, DELETE /api/reservations/:id	authenticate + authorize
Orders	POST /api/orders, GET /me, /:id, POST /:id/items, PATCH /:id/status	authenticate + authorize

6.1.3 Middlewares

- **auth.middleware.ts:** valida el JWT contra las claves públicas JWKS de Keycloak usando RS256 y adjunta la información del usuario a req.user.
- **role.middleware.ts:** verifica que el usuario tenga alguno de los roles permitidos antes de ejecutar la operación.
- **validate.middleware.ts:** valida el cuerpo de la solicitud mediante esquemas Zod y responde con error 400 cuando los datos no cumplen el contrato esperado.
- **cache.middleware.ts:** aplica una estrategia de caché bajo demanda con Redis. Si Redis no está disponible, la solicitud continúa sin interrumpir el servicio.
- **error.middleware.ts:** centraliza el manejo de errores no capturados y devuelve respuestas uniformes.

6.1.4 Configuración (apps/api/src/config/)

- env.ts valida todas las variables de entorno con Zod al arrancar. Si una variable obligatoria falta, el proceso termina con código 1. database.ts arma el pool de PostgreSQL; database.mongo.ts conecta Mongoose con replicaSet opcional; redis.ts crea el cliente solo si REDIS_URL está definida (caso contrario, getRedisClient() retorna null y la cache queda deshabilitada sin afectar al resto). keycloak.ts construye las URLs de token, JWKS y admin API a partir del realm.

6.1.5 Forma estándar de respuesta

Todas las respuestas pasan por sendSuccess o sendError, definidas en apps/api/src/utils/response.ts:

```
interface ApiResponse<T> {  
  success: boolean;  
  message: string;  
  data: T | null;  
}
```

Esto evita que distintos endpoints devuelvan formas distintas y facilita el manejo en el cliente.

6.1.6 Documentación interna OpenAPI

Swagger UI se expone en /api-docs cuando el servidor corre. La definición vive en apps/api/src/docs/swagger.json (OpenAPI 3.0.0) y describe los seis recursos junto con el esquema bearerAuth para JWT. De esta manera cumple con tener documentación interna intuitiva.

6.2 Microservicio de búsqueda y Elasticsearch

El microservicio de búsqueda separa las consultas de texto completo del núcleo transaccional. Está ubicado en `apps/search/`, se implementa con Express y TypeScript, y se ejecuta como un servicio independiente de la API principal. Su responsabilidad es recibir solicitudes bajo el prefijo `/search`, consultar Elasticsearch y mantener actualizado el índice de productos mediante un proceso de reindexación manual.

Esta separación evita que la API principal dependa directamente del cliente de Elasticsearch o de la lógica de construcción de índices. Si Elasticsearch o el microservicio de búsqueda presentan fallos, el impacto queda limitado a las rutas de búsqueda, mientras que las operaciones transaccionales bajo `/api` pueden continuar funcionando.

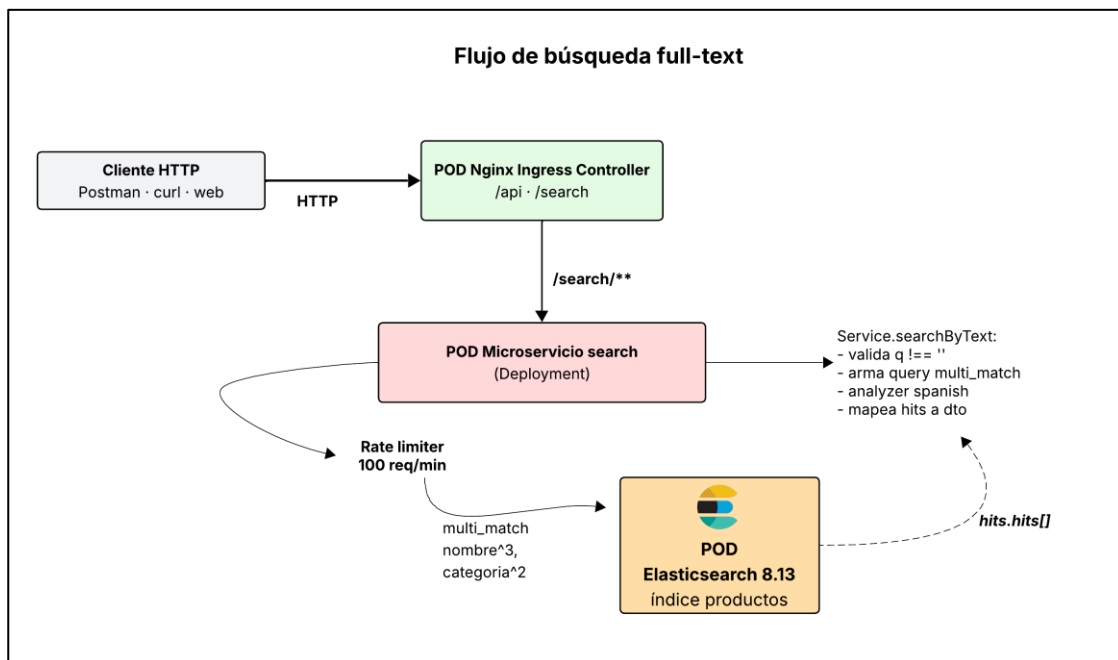


Figura 4. Flujo de una búsqueda full-text.

6.2.1 Responsabilidades

El microservicio Search es responsable de exponer los endpoints HTTP bajo el prefijo `/search` y atender las solicitudes relacionadas con búsqueda de productos. Su función principal es ejecutar búsquedas full-text sobre el índice `products` en Elasticsearch, aplicando consultas optimizadas para campos como nombre, categoría y descripción. Además, incorpora un rate limiting básico en las rutas de búsqueda para reducir el abuso de solicitudes repetitivas. Al iniciar, el servicio verifica la existencia del índice `products` y lo crea si todavía no existe. También se encarga del proceso de reindexación manual mediante `POST /search/reindex`: obtiene los ítems de menú desde la API principal usando `GET /api/menus/items/all`, transforma los datos recibidos al formato esperado por Elasticsearch y los inserta en el índice mediante operaciones bulk. De esta forma, Search mantiene a Elasticsearch como una proyección de lectura especializada, mientras la API principal continúa siendo la fuente de verdad del sistema.

6.2.2 Endpoints

Método	Ruta	Función
GET	/search/products?q=texto	Busqueda multi_match en nombre^3, categoria^2, descripcion (analyzer spanish).
GET	/search/products/category/:categoria	Filtro por categoria; usa el sub-campo keyword.
POST	/search/reindex	Borra el indice products y lo reconstruye con los items obtenidos desde la API.

6.2.3 Modelo de índice

Campo	Tipo	Notas
id	keyword	Identificador del item.
restaurantId	keyword	Para filtros por restaurante.
menuId	keyword	Para filtros por menu.
nombre	text (spanish)	Buscable, peso 3 en multi_match.
categoria	text + keyword multi-field	Buscable y filtrable. Peso 2 en multi_match.
descripcion	text (spanish)	Si el origen no la tiene, se guarda 'Producto sin descripcion'.
precio	float	Devuelto en los resultados.
disponible	boolean	Devuelto en los resultados.

7. Descripción general del flujo de datos y operación

El sistema sigue un flujo organizado por capas. Las solicitudes HTTP ingresan por NGINX Ingress, que actúa como punto único de entrada en Kubernetes. Según el prefijo de la ruta, el tráfico se redirige hacia la API principal cuando inicia con `/api`, o hacia el microservicio de búsqueda cuando inicia con `/search`.

Dentro de la API principal, una solicitud atraviesa la siguiente secuencia:

middlewares → controlador → servicio → DAO → motor de base de datos

Los middlewares resuelven aspectos transversales como autenticación, autorización, validación y caché. Luego, el controlador delega la operación al servicio correspondiente, donde se aplican las reglas de negocio. Finalmente, el servicio utiliza una interfaz DAO, cuya implementación concreta depende del valor de `DB_ENGINE`. De esta forma, la misma lógica de negocio puede trabajar con PostgreSQL o MongoDB sin modificar controladores ni servicios.

Redis interviene en las consultas frecuentes mediante una estrategia de caché bajo demanda. En una lectura cacheable, si existe una respuesta almacenada, se devuelve directamente desde Redis. Si no existe, la API consulta la base de datos y almacena la respuesta por un tiempo limitado. Elasticsearch participa únicamente en el flujo de búsqueda: el microservicio Search consulta el índice `products`, mientras que la API principal continúa siendo la fuente oficial de los datos.

Además del flujo de datos durante la operación del sistema, el proyecto cuenta con un flujo de integración y entrega continua mediante GitHub Actions. Cuando se generan cambios en el repositorio, el pipeline ejecuta validaciones automáticas como instalación de dependencias, revisión de estilo, pruebas unitarias y verificación de cobertura. Si las validaciones son correctas, el flujo de entrega construye las imágenes Docker de la API principal y del microservicio de búsqueda, y las publica en GitHub Container Registry. Este proceso no participa directamente en las solicitudes de los usuarios, pero forma parte del ciclo general de operación porque automatiza la validación, el empaquetado y la publicación de los servicios.

7.1 Uso del balanceador

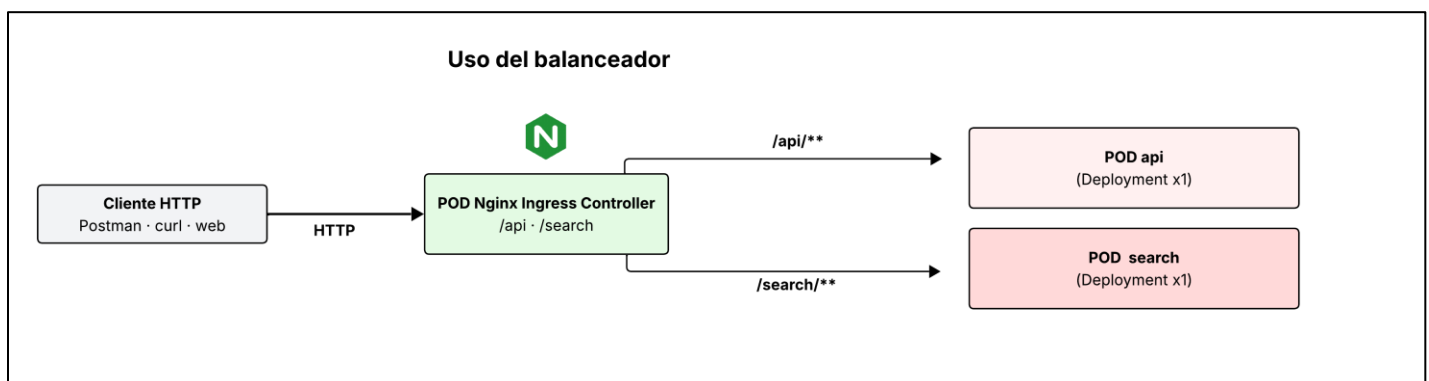


Figura 5. Flujo básico del uso del balanceador.

7.1.1 Nginx Ingress y balanceo

El punto único de entrada del sistema es el recurso `restaurant-ingress`, definido en `infra/k8s/common/ingress.yaml`. Este recurso pertenece al namespace `proyecto01-restaurante` y define las reglas de enrutamiento HTTP hacia los servicios internos de la aplicación.

El enrutamiento lo ejecuta el NGINX Ingress Controller, que se encuentra en el namespace `ingress-nginx`. De esta forma, el proyecto mantiene separadas las reglas propias de la aplicación y el componente de infraestructura que las interpreta.

7.1.2 Estrategia de routing

- `/api/**` → `api-service:80` → puerto interno 3000 → Deployment `api`.
- `/search/**` → `search-service:80` → puerto interno 3001 → Deployment `search`.

No se utiliza `rewrite-target` porque ambos servicios esperan recibir el prefijo completo de la ruta. La API registra sus rutas bajo `/api` y el microservicio de búsqueda registra sus rutas bajo `/search`. Si el prefijo se eliminara antes de llegar al servicio, las rutas definidas en Express no coincidirían.

Además, el recurso Ingress no define un host específico. Por esta razón, puede aceptar solicitudes dirigidas a `localhost` sin modificar el archivo `hosts` del sistema.

7.1.3 Diferencia entre Ingress y Ingress Controller

- **Ingress** es el recurso declarativo de Kubernetes. Define qué rutas existen y hacia qué servicios internos deben dirigirse.
- **Ingress Controller** es el componente que interpreta esas reglas y ejecuta el enrutamiento real. En este proyecto, ese controlador es NGINX.

Ambos elementos son necesarios: el recurso Ingress contiene las reglas, mientras que el Ingress Controller las aplica y permite que el tráfico llegue a los servicios correctos.

8. Representación del Cluster de Kubernetes

El clúster de Kubernetes utiliza principalmente el namespace `proyecto01-restaurante`, donde se agrupan los recursos propios del sistema: API principal, microservicio de búsqueda, Redis, Elasticsearch, Keycloak, PostgreSQL de Keycloak y el motor de base de datos de negocio seleccionado.

Además, existe un segundo namespace llamado `ingress-nginx`, que no pertenece directamente a la aplicación, sino a la infraestructura del clúster. En este namespace se ejecuta el NGINX Ingress Controller, componente encargado de leer las reglas definidas en el recurso Ingress del proyecto y materializarlas como enrutamiento HTTP hacia los servicios internos. Es decir, el recurso `restaurant-ingress` vive en `proyecto01-restaurante`, pero el controlador que lo interpreta y hace efectivo el balanceo se ejecuta en `ingress-nginx`.

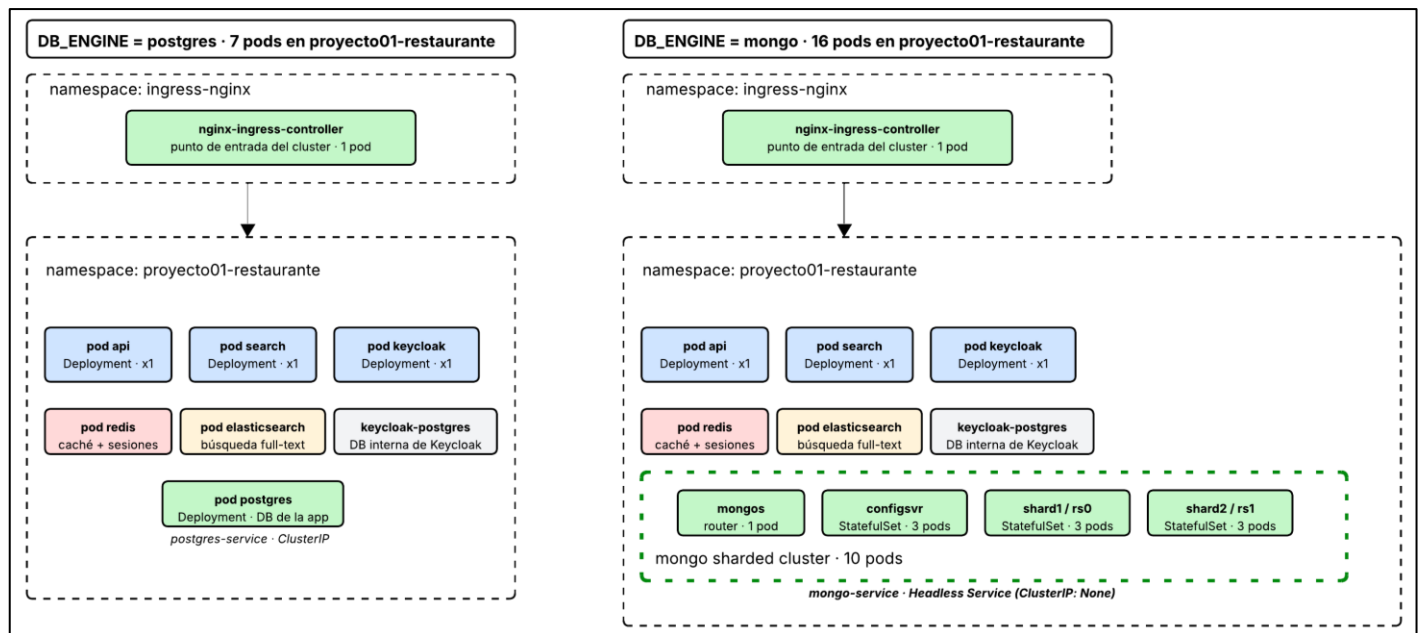


Figura 6. Diagrama de como se ve el cluster según el motor de base de datos.

La topología tiene un conjunto de recursos comunes que siempre se despliegan: la API principal, el microservicio de búsqueda, Redis, Elasticsearch, Keycloak, el PostgreSQL interno de Keycloak y el Ingress. El motor de base de datos de negocio se selecciona al ejecutar `deploy.ps1` con el parámetro `-DbEngine`. Si se elige mongo, se despliega un clúster MongoDB particionado compuesto por un replica set de configuración, dos shards y un router mongos, para un total de 10 pods. Si se elige postgres, se despliega una instancia única de PostgreSQL para la base de datos de negocio y no se levantan los recursos de MongoDB.

La API y el microservicio de búsqueda se ejecutan como Deployments, ya que no almacenan estado local y pueden escalar horizontalmente. Redis también se ejecuta como Deployment, porque funciona como caché efímera. En cambio, MongoDB, PostgreSQL y Elasticsearch se despliegan como StatefulSets, debido a que requieren identidad estable, almacenamiento persistente y volúmenes asociados.

Como mencionado anteriormente, el acceso externo al sistema ocurre por medio de NGINX Ingress. Desde ahí, las solicitudes se enrutan hacia los servicios internos: `/api` apunta a `api-service` y `/search` apunta a `search-service`. Los demás componentes permanecen accesibles únicamente dentro del clúster mediante servicios internos, lo que reduce la exposición directa de bases de datos, Redis, Elasticsearch y Keycloak.